

Functional Programming With **Scala**



Summary

1. A bit of History
2. What is scala ?
3. Principles of FP
4. Data modeling
5. Recursion and Data Structures
6. FP patterns



1. A bit of History

1936 : λ -calculus

- Introduced by Alonzo Church as a framework to express computation
- Minimal by design
- Turing complete

1936 : λ -calculus

- Variables : x
- Abstraction (or lambda) : $\lambda x.x$
- Application : $(\lambda x.x y)$

1936 : λ -calculus

- Rename variables
- Substitute variables

$$\begin{aligned} Y &:= \lambda g. (\lambda x. g (x x)) (\lambda x. g (x x)) \\ Y g & \\ & (\lambda h. (\lambda x. h (x x)) (\lambda x. h (x x))) g \\ & (\lambda x. g (x x)) (\lambda x. g (x x)) \\ & g ((\lambda x. g (x x)) (\lambda x. g (x x))) \\ & g (Y g) \end{aligned}$$

1936 : λ -calculus

- Church Numerals

$$0 := \lambda f . \lambda x . x$$
$$1 := \lambda f . \lambda x . f \ x$$
$$2 := \lambda f . \lambda x . f \ (f \ x)$$
$$3 := \lambda f . \lambda x . f \ (f \ (f \ x))$$

1936 : λ -calculus

- Church Numerals

$$0 \quad := \quad \lambda f . \lambda x . x$$
$$1 \quad := \quad (0 \quad (f \quad x))$$
$$2 \quad := \quad (1 \quad (f \quad x))$$
$$3 \quad := \quad (2 \quad (f \quad x))$$
$$S \, (n) \quad := \quad (n \quad (f \quad x))$$

1958 : LISP

- Designed by John McCarthy
- Inspired by λ -calculus
- Dynamically typed
- Representation of a program through lists and functions



1958 : LISP

- No operator

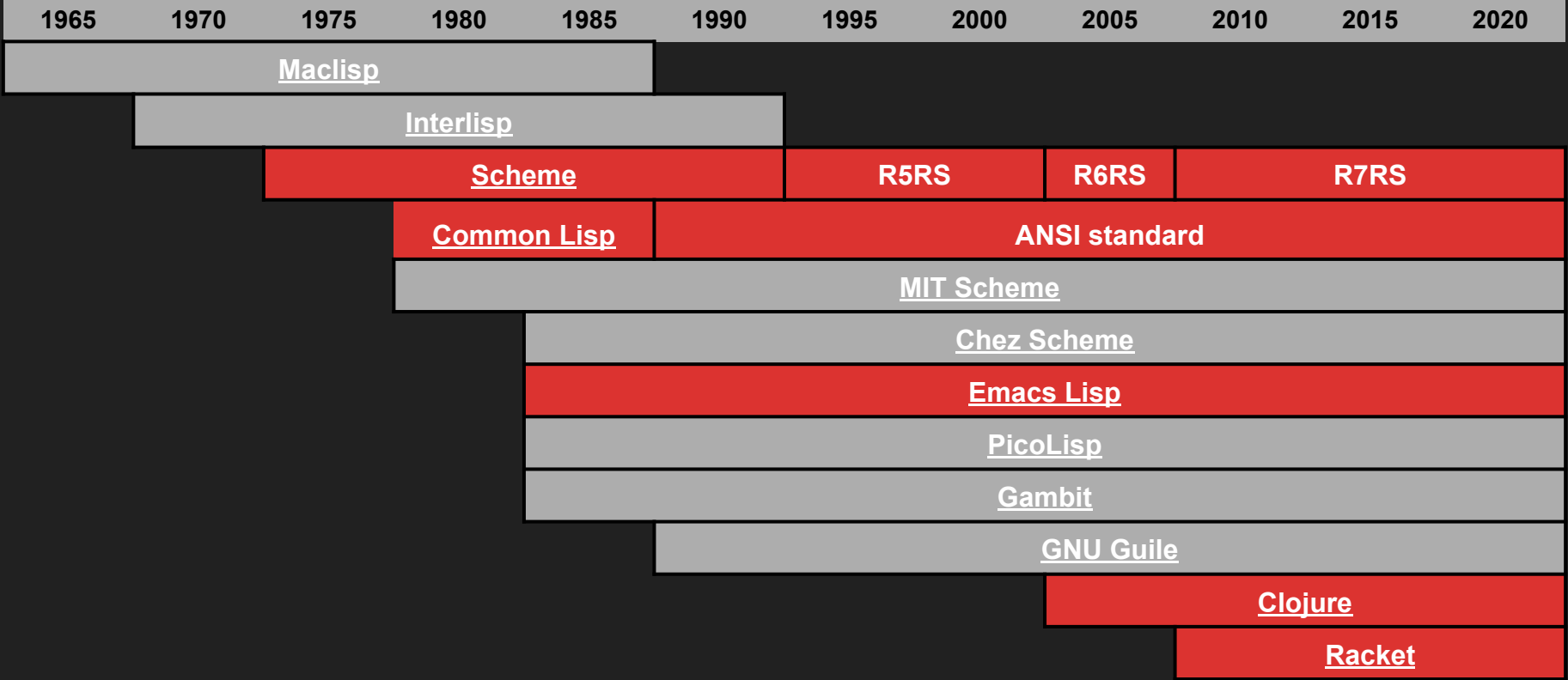
```
(+ 1 1)
```

- Anonymous functions

```
(lambda (x) (+ x 1))
```

- Really, everything is a list

```
(def increase (salary rate) (* salary (+ 1 rate)))
```



1958 : LISP

" In a homoiconic language, the primary representation of programs is also a data structure in a primitive type of the language itself. "

1973: ML

- Hindler-Milner static type system (inference)
- OCaml (1985) developed at INRIA
- Haskell (1990)
- Inspired a lot Scala design

1973: ML

```
fun fac (0 : int) : int = 1
  | fac (n : int) : int = n * fac (n - 1)
```

```
fun fac 0 = 1
  | fac n = n * fac (n - 1)
```

2003: Scala

*"2003 - A drunken Martin Odersky sees a Reese's Peanut Butter Cup ad featuring somebody's peanut butter getting on somebody else's chocolate and has an idea. He creates **Scala**, a language that unifies constructs from both object oriented and functional languages. This pisses off both groups and each promptly declares jihad."*

james-iry.blogspot.com/2009/05/brief-incomplete-and-mostly-wrong.html

Today: FP is everywhere

- OCaml is widely used in the academia via the CoQ proof assistant
- Jane's Street backends are full OCaml
- Facebook spam filtering engine is written in haskell
- Functional Reactive Programming is the basis for react's hooks
- Rust as a gateway ML for c++ devs
- **Scala** is the most mainstream FP language

3. What is scala ?

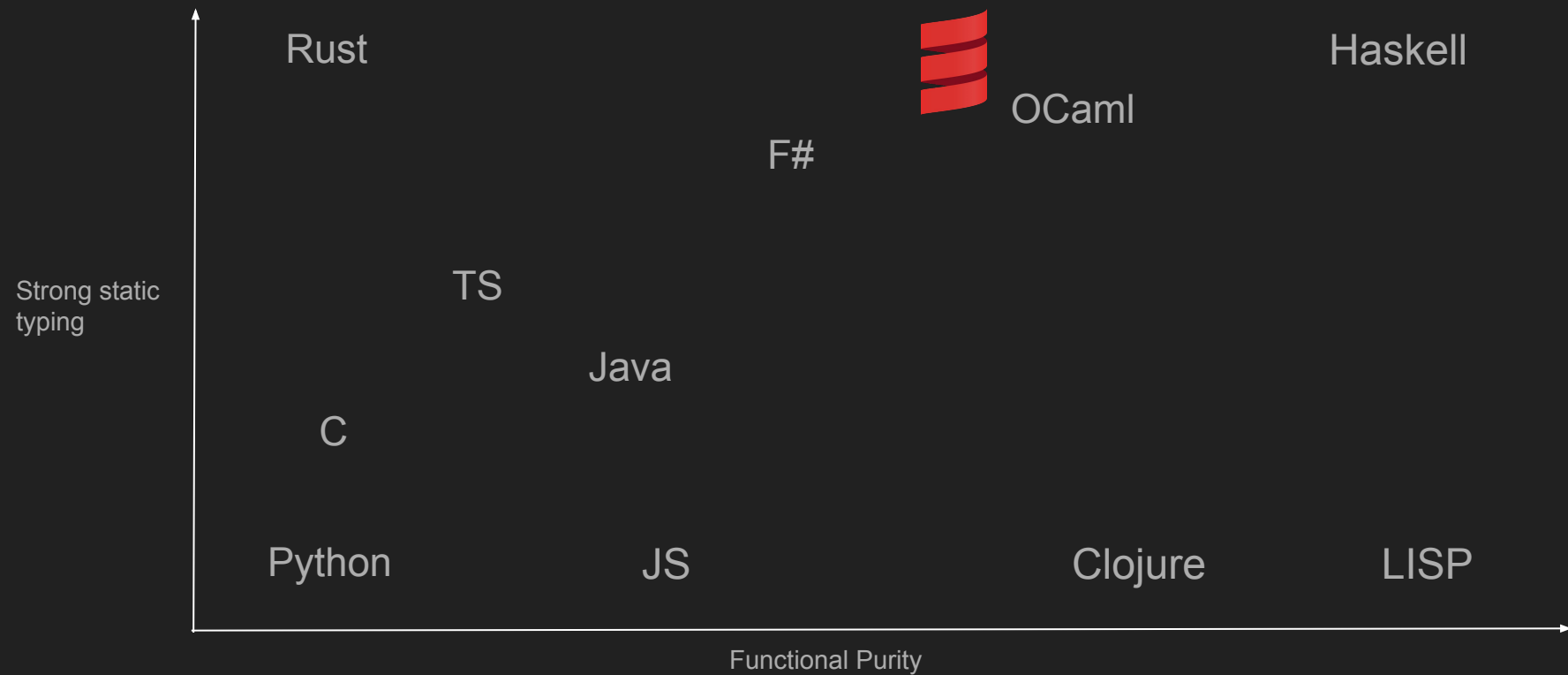
Scala Summary

Merges the OOP paradigm with the FP paradigm

"Objects for modules, functions for compute"

- Strong static type system
- Slick syntax
- excellent capabilities for abstraction.
- Great for Business logic modeling
- Extensive, functional collection library

Scala Summary



Scala Summary

Let's try it together !

<https://scastie.scala-lang.org/>

Scala Summary

Compilation Backends

- **JVM Bytecode**
 - Principal use case
 - Apache Spark
 - Backends
- **JS**
 - Full stack type safety
 - Tyrian, Laminar
- **Native code**
 - via LLVM and ScalaNative
 - Native library interop

3. Principles of FP

Function as a value

Function as value in Scala

// This is a usual function declaration in Scala.

```
def f(n: Int): Int = n + 1
```

// This is the same function using lambda expression syntax.

```
val f: Int ⇒ Int = n ⇒ n + 1
```

// This is the same function using lambda short syntax.

```
val f: Int ⇒ Int = _ + 1
```

Function as a value: example of refactoring

```
def multiplyBy4(values: List[Int]): List[Int] =  
  for (value ← values)  
  yield value * 4
```

```
def add26(values: List[Int]): List[Int] =  
  for (value ← values)  
  yield value + 26
```

```
def sub42(values: List[Int]): List[Int] =  
  for (value ← values)  
  yield value - 42
```

You want to apply different operations to a list of integers.

The problem is that you have to repeat the same block where you just change the operation.

This leads you to code reuse. It does not facilitate the maintainability of your project.

DRY (Don't Repeat Yourself).

Function as a value: example of refactoring

```
def multiplyBy4(values: List[Int]): List[Int] =  
  for (value ← values)  
  yield value * 4
```

```
def add26(values: List[Int]): List[Int] =  
  for (value ← values)  
  yield value + 26
```

```
def sub42(values: List[Int]): List[Int] =  
  for (value ← values)  
  yield value - 42
```

To avoid repetition here, you just have to put the operation in parameter in a single function.

```
def map(f: Int ⇒ Int)(values: List[Int]): List[Int] =  
  for (value ← values)  
  yield f(value)
```

Function as a value: example of refactoring

```
def multiplyBy4(values: List[Int]): List[Int] =  
  map(_ * 4)(values)
```

Then, you use the function.

```
def add26(values: List[Int]): List[Int] =  
  map(_ + 26)(values)
```

```
def sub42(values: List[Int]): List[Int] =  
  map(_ - 42)(values)
```

```
def map(f: Int ⇒ Int)(values: List[Int]): List[Int] =  
  for (value ← values)  
  yield f(value)
```

Why function as value?

```
def multiplyBy4(values: List[Int]): List[Int] =  
  map(_ * 4)(values)
```

```
def add26(values: List[Int]): List[Int] =  
  map(_ + 26)(values)
```

```
def sub42(values: List[Int]): List[Int] =  
  map(_ - 42)(values)
```

If, for a reason or another, you have to change the way you apply your operations, you only have to do it in one function only.

You do not have to apply the modification into multiple locations in your code, with the risk to miss one of them

```
def map(f: Int ⇒ Int)(values: List[Int]): List[Int] =  
  for (value ← values)  
  yield f(value)  
  yield if (value % 2 == 0) f(value) else value
```

Note on function as value

An extreme usage of function as value can lead to *one-liners*.

With method chaining, it is possible to write a whole program or a feature in just one line. But it also makes the code harder to read.

```
text.split("\n").toList.filter(_._trim.nonEmpty).map(_._toUpperCase).mkString("\n")
```

Do not hesitate to introduce lines breaks, use comments, and intermediate values.

```
val lines = text.split("\n").toList
lines
    .filter(_._trim.nonEmpty) // at least one non-space char in the line
    .map(_._toUpperCase)      // I like to scream
    .mkString("\n")
```

Function as value

"A function is a first-class citizen" $\Rightarrow$ a function can be manipulated like value of any kind.

- You can have a function that takes another function as parameter.
- You can have a function that returns a function.
- You can allocate a function to a variable.

Function as value is a property that helps to have a more concise and more readable code.

It is easier to maintain and to test.

Immutability

Immutability

A strong principle in FP is that the values in a program should not change.

In **Scala**

- the **val** keyword declares immutable **values**
- the **var** keyword declares mutable **variables**

Avoid: **var** a = 2; /* ... */ a = 4

Prefer: **val** a = 2; /* ... */ **val** b = 4

Nothing change, and this change everything!

Why immutability?

Mutability

- great for performance optimizations.
- hard to read and maintain (side effects, coupling)
- hard in case of concurrent programming.

Immutability

- hard to write (forced copying)
- sharing a reference of a state is safe
- concurrent programming is easier

Side effect

A side effect is an action produced by executing code that has an effect outside of its scope.

- you can't always guess at first sight what kind of effects a function is hiding.
- Often the cause of long sessions of on-call waking you up at 3AM

```
var x = 0
```

```
def totallyNotAffectingX(n: Int): Int =  
  x += 1  
  n + 1
```

```
totallyNotAffectingX(42) // 43  
totallyNotAffectingX(24) // 25  
assert(x == 0)           // Boom !
```

Side effect

Effect Catalog :

- modification of a global state
- IO actions (network calls, printing to the console, etc)
- Throwing Exceptions

```
def makeCoffee(beans: CoffeeBeans, water: Water): Coffee =  
    val powder    = grindCoffee(beans)  
    val hotWater  = heatWaterUp(water)  
  
    launchRocket() // TRIGGER A GLOBAL THERMONUCLEAR WAR  
  
    filterCoffee(powder, hotWater)
```

Example: mutable bank account

*// in this approach, the instance contains a mutable private state,
// that contains the last value.*

```
class BankAccount(initial: BigDecimal):  
  private var balance = initial  
  
  def withdraw(amount: BigDecimal): Unit =  
    balance -= amount  
  
  def deposit(amount: BigDecimal): Unit =  
    balance += amount  
  
  def balance: Double =  
    balance
```

Example: mutable bank account

```
val account1 = BankAccount(100)
account1.deposit(10)
```

```
// sharing the reference leads to side effect
```

```
val account2 = account1
account2.withdraw(20)
account2.withdraw(30)
```

```
println(account1.balance) // 60
println(account2.balance) // 60
```

Example: immutable bank account

*// in this approach, you have no mutable state, as balance below is a
// constant. Modifying here means create a copy of the instance with
// the new value.*

```
class BankAccount(balance: BigDecimal):  
  
    def withdraw(amount: BigDecimal): BankAccount =  
        BankAccount(balance - amount)  
  
    def deposit(amount: BigDecimal): BankAccount =  
        BankAccount(balance + amount)
```

Example: immutable bank account

// sharing the reference does not lead to side effect.

```
val account1 = BankAccount(100)
```

```
val account2 = account1.deposit(10)
```

// in addition, you can chain method calls

```
val account3 = account2.withdraw(20).withdraw(30)
```

```
println(account1.balance) // 100
```

```
println(account2.balance) // 110
```

```
println(account3.balance) // 60
```

Example: immutable data structure in Scala

Scala Collection API has two kinds of data structures

- `scala.collection.mutable`
- `scala.collection.immutable` < this is the default

```
// with List
val cities = List("Paris", "Madrid", "London")
cities.appended("Roma") // List(Paris, Madrid, London, Roma)
cities                  // List(Paris, Madrid, London)
```

```
// with Map
val ages = Map("Jon" → 32, "Mary" → 35)
ages.updated("Jon", 33) // Map(Jon → 33, Mary → 35)
ages                   // Map(Jon → 32, Mary → 35)
```

Note on immutability

The principle of immutability is not exactly true. In reality, your program respects the immutability principle, even if there is non-noticeable mutability inside. For example, in this program

```
val a = 2
val b = 4
a + b
```

It seems immutable, but in fact, the registers of the CPU will be modified (unless the compiler rewrites the expression into its result).

```
mov     edx, DWORD PTR [rbp-4]
mov     eax, DWORD PTR [rbp-8]
add     eax, edx
```

Note on immutability (1)

If performance is necessary, this code is acceptable, as it has no noticeable mutations perceivable outside its body (no side effect).

```
def sumOf(values: List[Int]): Int =  
  var sum = 0  
  for value ← values do sum += value  
  sum
```

As such, the function *sumOf* is considered as purely functional.

Not on the Note on immutability (1)

Performance oriented routines are usually already implemented on the standard library. Here we can use `foldLeft` to accumulate a result.

```
def sumOf(values: List[Int]): Int =  
  values.foldLeft(0): (sum, value) =>  
    sum + value
```

Or even better, if it is a list of a Numeric type :

```
def sumOf(values: List[Int]): Int =  
  values.sum
```

Note on immutability (2)

Immutability tends to make you create a huge quantity of small objects with a short span of usage.

- Extremely hard to get right with manual memory management (`malloc` / `free`)
- A Garbage collector (GC) make this trivial
- The JVM has tracing generational GCs which are *insanely good* at handling short lived objects

Please notice that the first GC *ever* has been created for LISP

Note on immutability (3)

As values do not change and as you can share references with no side effect, immutability is an interesting property in concurrent programming.

- No locks or barriers -> only one instance exists
- No locks therefore no deadlocks

Referential transparency

Referential transparency

*An expression is
referentially transparent
if it can be replaced
by its value
with no modification of the program behavior.*

Example

```
def countEven(l: List[Int]): Int =  
  l.count(_ % 2 == 0)
```

```
// we can certainly improve this expression  
countEven(List(1, 4, 3, 6, 8, 2, 3))  
  * (1 + 2 * countEven(List(1, 4, 3, 6, 8, 2, 3)))
```

Example

```
def countEven(l: List[Int]): Int =  
  l.count(_ % 2 == 0)
```

// we first introduce a variable to represent the list

```
val l = List(1, 4, 3, 6, 8, 2, 3)
```

```
countEven(l) * (1 + 2 * countEven(l))
```

Example

```
def countEven(l: List[Int]): Int =  
  l.count(_ % 2 == 0)
```

```
val l = List(1, 4, 3, 6, 8, 2, 3)
```

```
// then, we introduce another variable with the result of the function  
// application
```

```
val count = countEven(l)
```

```
// this expression is far more readable  
count * (1 + 2 * count)
```

Why referential transparency?

Referential transparency is one of the most important principles in FP.

FP is theoretically a declarative paradigm (ie. you give a problem and the computer solves it for you), where the expressions you write are rewritten, until you get a result.

This rewrite of expressions is only possible through the substitution model introduced by the property of referential transparency.

By using this substitution model, you can refactor your code to get a more readable content.

Counterexample

// This example shows that Random.nextInt is not referentially transparent.

```
import scala.util.Random
```

// Indeed, Random.nextInt is supposed a different value after each call.

// Below, there are 3 calls, so we get 3 different values

```
List(Random.nextInt(100), Random.nextInt(100), Random.nextInt(100))
```

// returns List(30, 42, 90)

// But here we only have one call. So, we get 3 times the same value.

```
val value = Random.nextInt(100)
```

```
List(value, value, value)
```

// returns List(77, 77, 77)

Fix counter-example

// We cannot really fix this example. The only way is to delay the execution of the random function.

```
import scala.util.Random
```

// We create a list of function returning a random value

```
List(() => Random.nextInt(100), () => Random.nextInt(100), () => Random.nextInt(100))
```

// then we call all those function

```
.map(f => f())
```

// returns List(30, 42, 90)

// We replace all the functions by a variable

```
val randomInt = () => Random.nextInt(100)
```

```
List(randomInt, randomInt, randomInt).map(f => f())
```

// So, we have the same behavior

// returns List(77, 90, 70)

Pure function

Determinism

A deterministic function always returns the same value for the same arguments.

```
// This function is deterministic.
```

```
def f(n: Int): Int = n + 1
```

```
// This function is not deterministic.
```

```
def f(n: Int): Int =  
    Random.nextInt(n)
```

Non-deterministic functions

```
class Signature(signer: String, timestamp: ZonedDateTime)
```

```
class Contract(terms: String):
```

```
class SignedContract(contract: Contract, signature: Signature)
```

```
// The function `sign` below is not deterministic, as each successive call to this  
// function with the same contract and the same person will return a different result,  
// unless you change the system clock.
```

```
def sign(contract: Contract, signer: String): SignedContract =  
    SignedContract(contract, Signature(signer, ZonedDateTime.now()))
```

Fixing Non-deterministic functions

```
// To have a deterministic function, it should not depend on the system clock and ensure  
// the timestamp is a parameter of the function.
```

```
def sign(contract: Contract, signer: String, at: ZonedDateTime): SignedContract =  
    SignedContract(contract, Signature(signer, at))
```

```
// Or simply depend on a signature now ... which make the function kind of useless
```

```
def sign(contract: Contract, signature: Signature): SignedContract =  
    SignedContract(contract, signature)
```

Non determinism is a dependency on non deterministic behavior (the operating system, an external service, or a probe).

Total function

A function f of type $A \Rightarrow B$ is total if f is defined for all elements of the type A .

Here, defined means that f does not throw an exception, fail, or have an infinite loop.

To make it total, one can use types like `Option` to signify the absence of value

```
// sqrt is a function that is only defined for positive values.  
// The problem is that the type Double represents both positive and negative values.  
// to avoid our function to return NaN (Not A Number), we indicate that it may not have  
// values with an Option.  
def sqrt(x: Double): Option[Double] =  
  if x ≥ 0.0 then Some(Math.sqrt(x))  
  else None
```

Total function

Another strategy is to craft a more precise type as input

```
class Positive(value: Double):  
  require(value ≥ 0)  
  
// Now this is total  
def sqrt(x: Positive): Positive =  
  Positive(Math.sqrt(x.value))
```

Pure function

A deterministic, total function is called a pure function.

- Pure functions are referentially transparent
- Pure functions are easy to refactor
- Pure functions are simple to test

Pure function

It is not possible for a program to be fully functionally pure. IO interactions are necessary to makes your application usable.

Most developers do in this case, they separate code into pure parts and impure parts. Side effects are then pushed to the frontier of the program.

We will talk about IO handling in FP later in this course.

Principles of FP : Key takeaways

The principles of FP provide you the tools to build reliable programs

- Function as value > better expressivity
- Immutability > avoid side effects
- Referential transparency > fearless refactoring
- Pure functions > consistency and testability

You end up with a program that *mostly* do what it is designed for.

4. Data Modeling

What is data modeling ?

- Encoding your domain model into structures and behavior
- No function writing involved at this stage
- Three steps :
 - What are the objects I want to manipulate in my model ?
 - Which one needs to be behaviours (open) or data structure (closed) ?
 - Which one will manipulate other objects or be manipulated ?

Encoding behaviours : traits

Traits (interfaces) are one of the most powerful tool in data modeling

```
trait Fighter:
  def attack: String
  def defend: String

class Warrior extends Fighter:
  def attack: String = "Me attack with sword"
  def defend: String = "Me defend with shield"

class Barbarian extends Fighter:
  def attack: String = "Me attack with big stick"
  def defend: String = "Defense is for weaklings"
```

Encoding behaviours : traits

Composition over inheritance

```
trait Healer:  
  def heal: String  
  
class Paladin extends Fighter, Healer:  
  def attack: String = "Me attack with sword"  
  def defend: String = "Me defend with shield"  
  def heal: String   = "Boom, full life"
```

Encoding behaviours : traits

Sealed trait for compile time safety

```
sealed trait Suit
```

```
object `♣` extends Suit  
object `♠` extends Suit  
object `♥` extends Suit  
object `♦` extends Suit
```

```
def isBlack(suit: Suit) = suit match  
  case `♣` | `♠` => true  
  case `♥`       => false  
  // Match may not be exhaustive, it will fail on ...
```

Product types and Sum types

- Product types are usually tuples / records / structures
 - If $D(x)$ is the domain of x , then $D(P) = D(p_0) \times D(p_1) \times \dots \times D(p_n)$ where $p_0, p_1 \dots p_n$ are the components of P
- Sum types represents enumeration and alternatives
 - If $D(x)$ is the domain of x , then $D(P) = D(p_0) \sqcup D(p_1) \sqcup \dots \sqcup D(p_n)$ where $p_0, p_1 \dots p_n$ are the components of P
 - $\sqcup$ is called the disjoint union, and may also be written as $+$, hence the name
- Algebraic data types (ADT)

Product types

- Scala has a Product trait, implemented by Tuples and case classes

```
val a: (String, Int) = ("foo", 123)
```

```
case class User(name: String, age: Int)
```

```
val b = User("Gandalf", 2587)
```

Sum types

- Sum types are implemented via sealed traits and enums

```
sealed trait Suit
```

```
case object `♣` extends Suit  
case object `♠` extends Suit  
case object `♥` extends Suit  
case object `♦` extends Suit
```

```
enum Suit:
```

```
case ♣  
case ♠  
case ♥  
case ♦
```

Sum types

- The two can be mixed to create powerful type hierarchies

```
trait Color
```

```
sealed trait Red extends Color
```

```
sealed trait Black extends Color
```

```
sealed trait Pointy
```

```
enum Suit:
```

```
case ♣ extends Suit, Black
```

```
case ♠ extends Suit, Black, Pointy
```

```
case ♥ extends Suit, Red
```

```
case ♦ extends Suit, Red, Pointy
```

Generic Types

- All higher order constructs : Classes, traits, enums can have type parameters
- Functions can have a type parameter list

```
case class Page[A](items: Seq[A], token: Option[String]):  
  
  def map[B](mapping: A => B): Page[B] =  
    Page(items.map(mapping), token)
```

Generic Types : type bounds

- Type may have bounds
- For all types : `Nothing <: A <: Any`

```
trait Spawnable[A]:  
  def make: A  
  
def spawnChild[A, B <: Spawnable[A]](parent: B): A = parent.make
```

Generic Types : variance

- Defines how subtyping relationships between types are preserved when their type parameters vary.
- Covariance $F[+A]$ preserves subtyping
If $B <: A$, then $F[B] <: F[A]$
- Contravariance $F[-A]$ invert subtyping
If $B <: A$, then $F[A] <: F[B]$

Generic Types : variance

Covariance $F[+A]$ preserves subtyping

If $B <: A$, then $F[B] <: F[A]$

```
case class Page[+A](items: Seq[A], token: Option[String])
```

```
trait JsonDecoder[+A]:  
  def decode(json: Json): Either[JsonError, A]
```

```
trait JsonEncoder[-A]:  
  def encode(value: A): Json
```

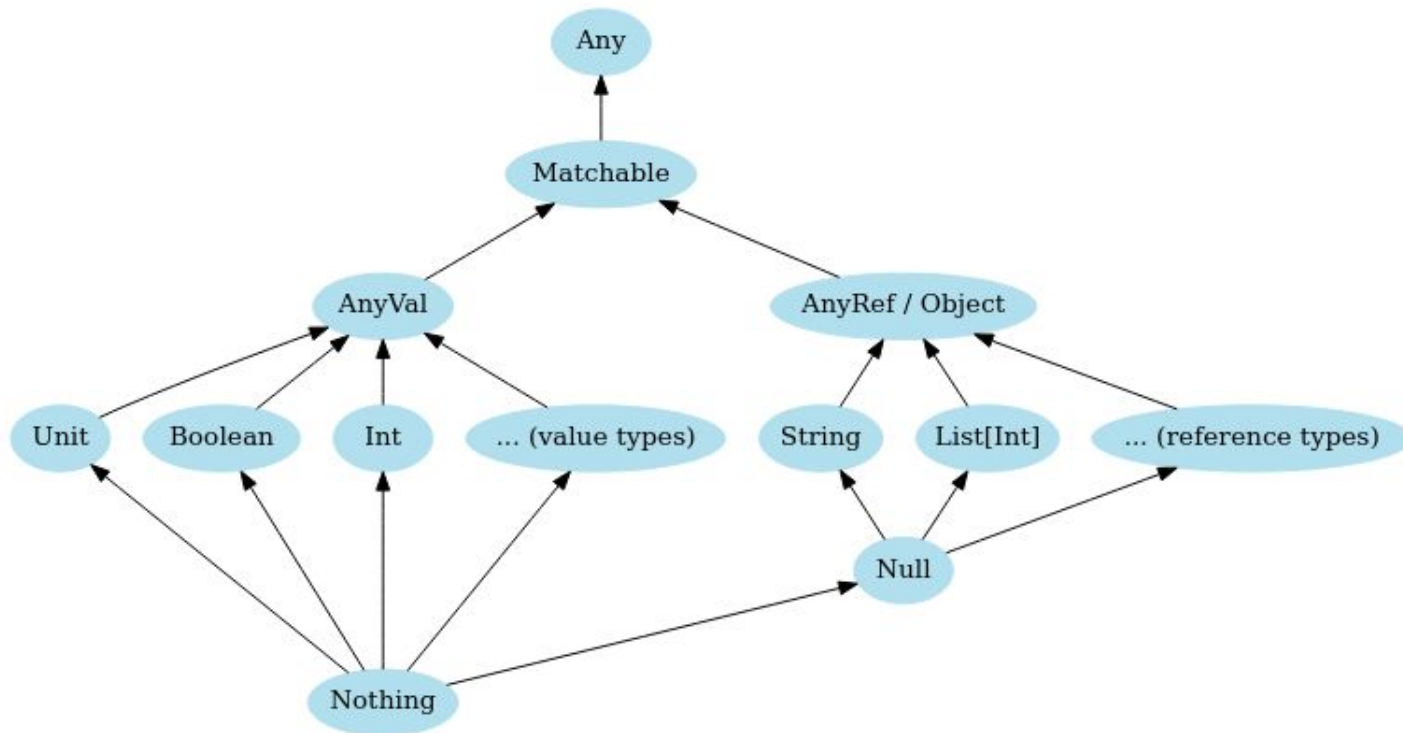
Generic Types : variance

Contravariance $F[-A]$ invert subtyping

If $B <: A$, then $F[A] <: F[B]$

```
trait JsonEncoder[-A]:  
  def encode(value: A): Json  
  
trait Function2[-A1, -A2, +R]:  
  def apply(a1: A1, a2: A2): R
```

Type hierarchy



5. Recursion and Data Structures

Recursion

- Recursion consists in solving a problem by working with :
 - structures that use :
 - structures that use :
 - structures that use :
 - structures that use :
- This is applicable to functions, but also to data structures and algorithms
- Finite way to represent infinity

Recursive function

Let's write the factorial function with a recursive style.

```
def fact(n: Int): Int =  
  if (n ≤ 1) 1  
  else n * fact(n - 1)
```

- recursive functions fill the call-stack up.
- StackOverflowError.

Tail recursive function

```
def fact(n: Int): Int =  
  @tailrec  
  def go(n: Int, acc: Int): Int =  
    if n ≤ 1 then acc  
    else fact(n - 1, n * acc)  
  go(n, 1)
```

- This is called tail recursion.
- This function can easily be converted into a while-loop.

Notice: in Scala, to ensure your function is tail recursive, you can add the annotation `@tailrec`. It will be checked during compile-time.

Tail recursive function

```
def fact_optim(n: Int): Int =  
  var acc = 1  
  var i = n  
  while i > 1 do  
    tmp = tmp * i // acc * n  
    i = i - 1     // n - 1  
  tmp
```

- no call-stack = no StackOverflowError
- you can still have an infinite loop

Scala performs this optimization at the bytecode level during compile-time.

Note on recursive function

Even if recursion is a fundamental part of FP, it is rarely used by developers, as it can produce bugs. This happens, especially if you mistype the stop-condition or the recursive call.

- *Recursion is the assembler of functional programming*
- Specialization is less error prone : *map, filter, fold*

```
def fact(n: Int): Int = (1 to n).product
```

Toolkit

The standard lib offers many convenience methods (go and explore it !)

Here are some examples for Seq[A]

<code>.map[B](f: A ⇒ B): Seq[B]</code>	transform
<code>.filter(p: A ⇒ Boolean): Seq[A]</code>	filter out elements
<code>.flatMap(f: A ⇒ Seq[B]): Seq[B]</code>	transform then flatten
<code>.foldLeft[R](z: R)(f: (R, A) ⇒ R): R</code>	accumulate
<code>.collect[B](pf: PartialFunction[A, B]): Seq[B]</code>	map + filter + patterns
<code>.groupBy[K](f: A ⇒ K): Map[K, Seq[A]]</code>	group elements

Recursive data structure

The most basic data structure in FP is the linked-list, which is defined in a recursive way.

```
enum LList[+A]:  
  // represents the end of the linked list  
  case LNil  
  // this is a constructor that adds an element at the beginning of the list  
  case Cons(value: A, tail: LList[A])  
  
import LList.*  
  
// represent the list [1, 2, 3]  
Cons(1, Cons(2, Cons(3, LNil)))
```

Recursive data structure

- They are everywhere
 - Query plans
 - Abstract syntax trees
 - JSON
- FP is generally very good at it
- Memoization

6. FP Patterns

FP Patterns 1 : Handling missing elements

```
enum Option[+A]:  
  case None extends Option[Nothing]  
  case Some(value: A) extends Option[A]
```

- Represent an optional element
- Is *technically* a Seq[A]
- No more NullPointerExceptions

FP Patterns 2 : Handling errors

```
case class IBAN(content: String)
case class IBANFormatError(message: String) extends Exception(message)

def unsafeParsing(input: String): IBAN =
  if input.length < 20 then throw IBANFormatError(s"IBAN input is too short: $input")
  if input.length > 32 then throw IBANFormatError(s"IBAN input is too long: $input")
  IBAN(input)
```

- Throwing exceptions break the referential transparency
- Throwing exceptions makes the function not total

FP Patterns 2 : Handling errors

```
case class IBAN(content: String)
case class IBANFormatError(message: String) extends Exception(message)

def optionParsing(input: String): Option[IBAN] =
  if      input.length < 20 then None
  else if input.length > 32 then None
  else  Some(IBAN(input))
```

- One can use options to modelize faillible functions
- Not allowing to know what happened

FP Patterns 2 : Handling errors

```
case class IBAN(content: String)
case class IBANFormatError(message: String) extends Exception(message)

enum Either[+L, +R]:
  case Left(value: L) extends Either[L, Nothing]
  case Right(value: R) extends Either[Nothing, R]

def eitherParsing(input: String): Either[IBANFormatError, IBAN] =
  if input.length < 20 then Left(IBANFormatError(s"IBAN input is too short: $input"))
  else if input.length > 32 then Left(IBANFormatError(s"IBAN input is too long: $input"))
  else Right(IBAN(input))
```

- Either allow use to bring context
- Try[A] as Either[Throwable, A]

FP Patterns 3 : Handling state

```
class Field

class Document:
  def add(field: Field): Unit = ()
  def send(to: String): Unit =
    throw Exception("DocumentAlreadySent")

val doc      = Document()
val commons = List(Field(), Field(), Field())

commons.foreach(doc.add)

// Send the document
doc.send("Alice")

// Later in the codebase
doc.add(Field())
doc.send("Bob") // Boom !
```

FP Patterns 3 : Handling state

```
enum DocumentBuilder:  
  case Init  
  case Step(builder: DocumentBuilder, field: Field)  
  
private def build: Document =  
  this match  
    case Init =>  
      Document()  
    case Step(builder: DocumentBuilder, field: Field) =>  
      val current = builder.build  
      current.add(field)  
      current  
  
def send(to: String): Unit =  
  this.build.send(to)
```

FP Patterns 3 : Handling state

```
val commons = List(Field(), Field(), Field())

val doc = commons.foldLeft(DocumentBuilder.Init)(DocumentBuilder.Step.apply)

// Send the document
doc.send("Alice")

// Later in the codebase
val doc2 = DocumentBuilder.Step(doc, Field())
doc2.send("Bob") // Works fine
```

- State manipulation can be expressed very safely with DSL
- Code = Data is a powerful tool
- can be generalized later

FP Patterns 4 : type classes

Scala has a implicit parameter resolution engine

```
given Int = 3  
  
def useInt(using i: Int): Int = i  
  
useInt // 3
```

FP Patterns 4 : type classes

Implicits can depend on other implicits

```
trait Foo:
  def foo: Int

trait Bar:
  def bar: Int

given Foo with
  def foo = 4

given (using foo: Foo): Bar with
  def bar = foo.foo
```

FP Patterns 4 : type classes

Deduction works on generic parameters

```
trait Printer[A]:  
  def print(a: A): String  
  
given Printer[Int] with  
  def print(a: Int): String =  
    s"Int($a)"  
  
given Printer[String] = identity  
  
given [A](using inner: Printer[A]): Printer[Option[A]] with  
  def print(a: Option[A]): String = a match  
    case Some(value) ⇒ s"Some(${inner.print(value)})"  
    case None        ⇒ "None"
```

FP Patterns 4 : type classes

Type classes are a way to handle *ad-hoc polymorphism*

```
trait Printer[A]:  
  def print(a: A): String  
  
def showUsThePrint[A](a: A)(using printer: Printer[A]): Unit =  
  println(printer.print(a))  
  
def showUsThePrint2[A: Printer](a: A): Unit =  
  println(summon[Printer[A]].print(a))
```

FP Patterns 4 : type classes

Type classes are a way to handle *ad-hoc polymorphism*

```
trait Functor[F[_]]:  
  def map[A, B](fa: F[A])(f: A ⇒ B): F[B]  
  
trait Semigroup[A]:  
  def combine(l: A, r: A): A  
  
trait Monoid[A] extends Semigroup[A]:  
  def empty: A
```